

ME500**ML/AI Hands-On Day 1****Connecting Atomistic Simulation Data to Experiment with Machine Learning****What you are doing today**

This is your first opportunity to combine the ML theory and workflow-design ideas from the previous lectures into a realistic materials-science problem. You will work as a group, make your own workflow decisions, implement the analysis, and defend the scientific conclusions that follow from it.

The two challenges

- Chapter 1 is the default challenge. Every group should begin there: use ML to improve an imperfect atomistic prediction of an experimental mechanical property.
- Chapter 2 is an optional extension for groups that finish early or want a harder problem: use labeled simulated structural fingerprints to interpret unlabeled experimental-like measurements.

The goal is not to guess the instructor's algorithm. The goal is to build a defensible workflow.

Student guide for in-class group work

How to Use the Provided Scripts

The scripts are designed to remove routine programming friction without solving the ML problem for you. **Run the student script once before changing anything.**

Already provided	Synthetic data generation, safe file paths, automatic output directories, CSV loading, basic NumPy/Pandas helpers, figure saving, model saving/loading, and result-table writing.
Your responsibility	Problem formulation, feature/target construction, data preprocessing, train/test design, baseline definition, model choice, training, testing, and scientific interpretation.
Generated folders	data/ contains the classroom dataset; figures/ is for plots; models/ is for trained models; results/ is for metrics and prediction tables.
Important rule	Do not use the experimental answer as an input feature for a prediction that is supposed to recover that same experimental answer.

A useful mindset

Treat the generated CSV files as if a collaborator handed you simulation and experimental data. The equations that generated the synthetic data are infrastructure, not clues you are expected to reverse engineer.

A Suggested 90-Minute Rhythm

1. First 10 minutes: run the script, inspect the data, and agree on what scientific question you are actually solving.
2. Next 15 minutes: sketch your proposed workflow before coding. Decide what the model sees, what it predicts, what your baseline is, and what counts as unseen test data.
3. Next 35 minutes: implement the preprocessing, training, and testing workflow.
4. Next 20 minutes: diagnose performance, create a clear comparison figure, and inspect failure cases.
5. Final 10 minutes: prepare a short group explanation of what you did, why you did it, and when you would or would not trust the result.

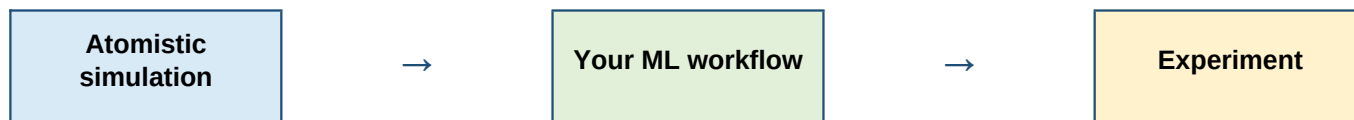
If you finish Chapter 1 early

Do not simply add a larger model. First ask whether you can make the evaluation more rigorous, identify a failure mode, or explain why the correction succeeds. Then move to Chapter 2 if time remains.

CHAPTER 1

Can ML Improve an Imperfect Atomistic Model?

Default challenge. Every group should begin here.



The Problem

Atomistic simulations can predict mechanical properties, but the prediction is only as good as the underlying physical model. Approximations in interatomic interactions, limited model flexibility, or physical effects missing from the simulation can create systematic disagreement with experiment.

You have a dataset of 320 synthetic materials. For each material, the supplied script provides atomistic descriptors and simulated mechanical properties, together with an experimental Young's modulus. The raw simulated Young's modulus is useful, but intentionally imperfect.

Simulation-derived descriptors	Cohesive energy, atomic volume, coordination number, mean bond length, simulated bulk modulus, simulated shear modulus, and simulated Young's modulus.
Experimental quantity	Experimental Young's modulus.
Scientific tension	The simulation contains real predictive information, but its error is not purely random.
Your question	Can ML use information available from the atomistic calculation to make the simulation more useful when predicting experiment?

Why This Matters

- Simulation is often much cheaper, faster, or easier to repeat than experiment, but simulation and experiment rarely agree perfectly.
- A systematic simulation error can contain information. If that error changes with bonding, structure, or other physical descriptors, a data-driven model may be able to learn part of the discrepancy.
- ML is only useful if it improves a meaningful baseline on data that were not used to train the model.
- A model that produces a smaller training error but fails on new materials has not solved the scientific problem.
- This same general idea - using ML to connect an imperfect physical model with experiment - will reappear later in the semester with more sophisticated simulation methods.

CHAPTER 1 - CONTINUED

Your Goals

By the end of this challenge, your group should be able to show that you can:

Core scientific idea

You are not trying to replace physics with ML. You are asking whether the simulation already captures enough physics that ML can learn the remaining structured disagreement with experiment.

- Formulate the simulation-to-experiment problem as an ML task.
- Identify which columns are legitimate inputs and which quantity or quantities should be predicted.
- Create a defensible training and testing strategy.
- Establish a non-ML baseline using the information already available from the atomistic simulation.
- Perform any preprocessing required by your chosen model without leaking information from the test set.
- Train a model using only the training information.
- Evaluate the final workflow on held-out materials.
- Compare the ML-assisted result against the raw simulation using quantitative metrics and a clear scientific figure.
- Identify at least one circumstance in which the ML-assisted prediction should not be trusted.

What counts as success?

A successful result is not "my model has a small error." A successful result is "on genuinely unseen materials, my ML-assisted workflow improves on a clearly defined baseline, and I can explain the assumptions and limitations behind that improvement."

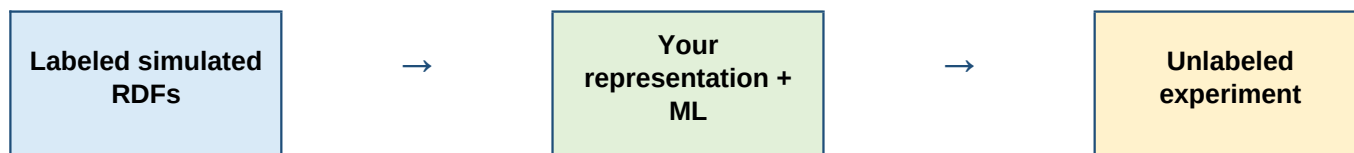
Where to Start

1. Run `student_template.py` without modifying it. Confirm that the dataset is created and read the printed column names and first few rows.
2. Inspect the relationship between the simulated Young's modulus and the experimental Young's modulus. This raw comparison is the performance your ML approach must beat.
3. Decide what you want the ML model to learn. Should it predict the experimental property directly, or should the existing atomistic prediction play a more explicit role in the formulation?
4. List the candidate input features and ask a simple question for every one: would this quantity be available before the experimental answer is known?
5. Inspect feature ranges and distributions. Decide whether the algorithm you choose requires scaling, transformation, or other preprocessing.
6. Decide how you will reserve truly unseen samples for final testing before you fit the model.
7. Only after these decisions are written down should you implement the training workflow.

CHAPTER 2

Which Simulated Atomic Structure Matches Experiment?

Optional advanced challenge. Move here after Chapter 1 if your group has time and wants a more difficult simulation-to-experiment problem.



The Problem

Experiments often measure indirect structural signatures rather than an atomic configuration itself. A radial distribution function, or RDF, describes how likely it is to find neighboring atoms at different distances. Different structural states produce different RDF patterns, but experimental measurements can be broadened, shifted, rescaled, and noisy.

The simulation library contains 280 labeled RDFs: 70 examples from each of four structural states. You also have 12 experimental-like RDFs whose structural labels are hidden. Each curve is sampled at 110 radial positions.

Candidate states	Crystal, defect-rich crystal, amorphous, and liquid.
Simulation data	Labeled RDF curves generated from simulated structural states.
Experimental-like data	Unlabeled RDF curves with additional peak broadening, shifts, background variation, intensity changes, and noise.
Your question	Can a model trained on simulated structural fingerprints identify which simulated state is most consistent with each experimental-like measurement?

Why This Matters

- Atomistic simulations provide structures with known physical conditions, while experiments often provide signals that must be interpreted.
- A labeled simulation library can act as a reference space for experimental measurements.
- The difficult part is not merely classifying simulated data. The real challenge is transferring what was learned from the simulation domain to the experimental domain.
- Small differences between simulated and experimental signals can cause a model with excellent simulated test performance to fail on real measurements.
- This problem introduces a central theme of materials informatics: deciding whether experiment lies inside the physical/data domain represented by your simulations.

CHAPTER 2 - CONTINUED

Your Goals

By the end of this challenge, your group should be able to:

- Identify which columns constitute the RDF fingerprint and which columns are metadata or labels.
- Visualize representative curves before choosing a model.
- Create an honest train/test evaluation using only the labeled simulated data.
- Decide whether the raw 110-point curves should be used directly or transformed into another representation.
- Choose and train a classifier appropriate for the dataset and representation.
- Apply exactly the same fitted preprocessing and representation to the experimental-like data.
- Produce predicted structural states for the experimental-like samples.
- Create a visualization that helps you judge whether the experimental samples resemble the simulated training domain.
- Explain why a correct simulated test result and a trustworthy experimental interpretation are not the same thing.

What counts as success?

A successful workflow first demonstrates that it can generalize to unseen simulated structures. It then uses that validated workflow to interpret experiment while explicitly discussing whether the experimental-like curves are represented well by the simulation library.

Important warning

Excellent accuracy on held-out simulated RDFs is necessary, but it is not proof that your experimental predictions are correct. Simulation-to-experiment domain shift is part of the scientific problem.

Where to Start

1. Run `student_template.py` without modifying it. The script will generate the simulated and experimental-like RDF files and save a preview plot of simulated curves.
2. Look at several RDFs from each labeled state. Ask what physical differences in peak positions, peak widths, and longer-range structure appear to distinguish the classes.
3. Identify the feature columns that actually contain $g(r)$ values and separate them from sample identifiers and simulation labels.
4. Decide how you will create an honest simulated train/test split. The experimental-like samples should not be used to make the classifier look better.
5. Decide whether the raw RDF coordinates are an appropriate representation for your chosen classifier or whether some preprocessing or dimensionality reduction would be useful.
6. Train and test only on the simulated data first. Do not move to the experimental-like samples until you know how well your method handles unseen simulations.
7. Apply the same fitted workflow to the experimental-like RDFs and save the predicted states.
8. Use plots as a scientific diagnostic, not just decoration. Ask whether the experimental-like samples appear to occupy the same structural space as the simulations.

Before You Start Coding: Five Questions for Every Group

These questions matter more than the specific ML algorithm you choose.

1. What is the scientific question?

State it in one sentence. If your group cannot agree on this, do not start coding yet.

2. What information is available at prediction time?

Features should represent information you could realistically know before the quantity or label you are trying to predict.

3. What is the baseline?

ML should be compared with the simplest scientifically meaningful approach, not with nothing.

4. What data are genuinely unseen?

Your final test should answer the question you actually care about, and preprocessing should not leak information from that test set.

5. What would make you distrust the result?

A scientific ML workflow should identify its failure modes and domain of applicability, not only its average accuracy.

End-of-class group summary

Be prepared to explain your workflow in approximately two minutes: the question you solved, the baseline you used, the choices you made, the performance you obtained on unseen data, and one limitation or failure mode you identified.

Build the simplest workflow that can answer the scientific question convincingly.